

Fast Graph Refinement and Implicit Neural Representation for Tissue Tracking

Adam Schmidt¹, Omid Mohareri², Simon DiMaio², and Septimiu E. Salcudean¹

Abstract—Tracking of tissue in the surgical environment is often done via locating frame-to-frame keypoint correspondences, and then using these correspondences to warp a prior underlying model such as a spline, mesh, or embedded deformation. We introduce a novel learned model which takes keypoint correspondences as input and enables a prior-free estimation of deformation at any location. For fast point tracking, our model allows for sparse queries, unlike dense grid based CNNs, which run on full images. Our model begins with a novel graph-based point refinement scheme which refines matched keypoints, updating their features and movement instead of discarding possible outliers. Then, we use these refined matches to learn a novel neural implicit representation for estimating movement of any location given its k-nearest neighbor (k-NN) keypoints. We name our implicit deformation model KINFlow (k-NN implicit neural flow). We demonstrate the performance of KINFlow photometrically on three different datasets. KINFlow is the first model to use a graph network to estimate flow of arbitrary query points, and can estimate movement of 1024 points in under 3 ms.

I. INTRODUCTION

Force estimation [1], motion compensation [2], image guidance, scene understanding [3], surgical perception [4], and navigation [5] all benefit from tissue tracking. Researchers have studied this problem extensively [6]–[10], with methods changing along with increasing computing and modelling capabilities. Tissue tracking also enables the reconstruction of a scene with model, or backprojected surfels [4], [11]. Additional effort has addressed surgical tool masking as this is prior information we know is separate from the tissue [12]. Approaches still use underlying models for deformation, constraining estimable movement to said model. Ideally, we could learn this deformation model as well. We look to learn a general method that can automatically track how features move between frames without any prior information or ground truth. This method should be able to track surgical tissue, and instrument motion in the surgical sequences. Our method learns to refine outlier matches into features that are usable. Then, it uses these refined matches to estimate flow for any point in the image. Our method is the first method to learn and model the transformation of a point as a deformed transformation of its neighbors, or relatives/kin. We call our method KINFlow (k-nearest neighbor implicit neural flow) for this reason. KINFlow will enable robust tracking of points without requiring calculating flow for full images, efficiently reusing

correspondences that will also be used for tracking and mapping.

Currently in the surgical space, there is large reliance on image descriptors for tracking. In separate fields, optical flow, RAFT [13], and learned features for cost volume correspondence are often used for estimating flow. These require estimating a cost volume for every possible pixel pair, which can be expensive, thus downscaling features to 1/8 resolution. We would like to use the computations we can use in SLAM or place recognition for fast tracking of arbitrary points on surgical tissue, and thus we choose to continue using keypoints. To do so, we look to implicit neural representations, a recent direction in machine learning, where we take in a query point as an input, and use a network to evaluate a function at the query point. This was initially done for radiance fields (querying rays) [14], and then on images (querying pixels) [15], [16], and light fields [17]. Extensions have been proposed for conditioning fields on images [18], enabling dynamic scenes [19], estimating pose [20], reconstructing RGB-D surfaces [21], estimating a global deformation field [22], and even for simultaneous localization and mapping [23].

We believe that by using already-calculated keypoints, we can learn an implicit tracking function that does not rely on resolution, priors, or control points. This is important in our case as cutting, movement of instruments, and blood can not be fully accounted for by limiting movement to a fixed model such as embedded deformation or a spline. One way to deal with this for instruments is to use masks [4], [12], but changes in tissue images are also important and cannot be addressed by using masks. KINFlow tracks motion of both tissue and instruments without requiring a separate solver for splines or embedded deformation.

We choose to refine outliers instead of discarding them because it allows us to keep relevant feature data along with requiring less computation. Networks such as SuperGlue [24] look to estimate inlier matches and discard outliers. We decide to refine all matches and ideally move them all to become usable as inliers. We then learn the flow as a function of neighboring refined keypoints and their learned features. We only have to filter and refine the full graph once per frame, and we can query flow at any arbitrary query location. Our method is independent of resolution, and could even be used to estimate at a resolution higher than image resolution, as we can sample flow at any real-valued location.

Currently, works are limited by underlying models, along with computational costs of integrating networks into interactive implementations. In our work, we introduce KINFlow,

This work was supported by Intuitive Surgical

¹Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC V6T 1Z4, Canada {adamschmidt, tims}@ece.ubc.ca

²Advanced Research, Intuitive Surgical, Sunnyvale, CA 94086, USA

a novel means to estimate flow in a fast and flexible way without requiring prior model knowledge or labelled data. KINFlow is the first method to use the neighbors of a point to generate an implicit flow field, and will enable applications to track points flexibly. KINFlow is very cheap for tracking few points, and scales linearly as points are increased. In Section II we will cover related works in surgical tracking, graph networks and implicit functions. Section III covers our model design and implementation. Our experiments, generalization, and benchmarking are covered in Section IV. We finally conclude in Section V.

II. RELATED WORKS

Our method uses sparse matches to estimate flow of the surgical tissue. Some relevant works use sparse matches for similar applications such as depth mapping or optical flow, but they do not do so with a neural implicit representation. Instead, they conform to an integer grid or use a CNN instead of a graph representation [25], or are still dense for finding top correspondences [26]. In [27], sparse matches are used as information to create a correspondence vector field, but this is not real-time viable. Closer work uses 3D CNNs for estimating RGB-D embedded deformation models [28], [29], but we want to stay in the graph for efficiency.

Point Clouds and Graphs: Networks that deal with learning operations on graphs like PointNet [30] and PointNet++ [31] provide good inspiration, but they result in a global estimate while we look for an estimate that varies per point. The grouping operation in PointNet++ is similar to how we choose to condition a point's flow estimate by using its neighbors. The dynamic graph CNN model [32], which treats a point cloud as a graph, pools along edges with features based on distance. This is another means to run convolution on a graph, but it pools via methods that smooth and lose spatial information such as a sum. In the graph world there are ways to pool using attention [33] to weight contributions between edges instead of just combining all features evenly [34], [35]. These help avoid over-smoothing and we choose to use these for our graph operations. For signed distance function reconstruction, Points2Surf [36], and NeuralIMLS [37] sample local regions to estimate a signed distance function, but without a graph operation beforehand to allow far-range message passing. Finally, there is research on representation of surfaces and meshes using Graph Convolutional Networks [38], but we do not want to force a mesh representation.

Surgical Tracking: For surgical applications of tissue tracking, Giannarou et al. [1] estimate deformation using a thin plate spline and affine features [9] for estimation of tissue and tool interaction to provide surgeon feedback. Yip et al. [39] track rigid regions using salient features [8] in them. Zhou et al. [3] create a 2D mosaic of the scene. In 3D tracking and mapping, Zhan et al. [40] look to automate tissue scanning and use classical descriptors for regions. SD-DefSLAM [41] moves SLAM to the deformable field, modelling deformation using a triangular mesh.

We do not want to constrain the deformation, thus KINFlow uses a query's nearest neighbor correspondences to parameterize an implicit neural representation. Our work is different from those mentioned as we use a graph representation to learn a deformation function for a point relative to its neighbors.

III. METHODS

Our model takes in a set of unfiltered correspondences between images I and I' . These correspondences can be computed using an off-the-shelf classical detector and a descriptor like SIFT [42]/SURF [43]/ORB [44], or with a more modern learned descriptor. We use the ReTRo [45] descriptor trained on our dataset, but any learned descriptor that can train with pose supervision would work [46], [47]. We use this to find N initial correspondences by detecting and describing points using the chosen descriptor and picking the nearest features in space. Each correspondence is a pair of points, denoted as (p_i, p'_i) , with p_i as the point's position in I , and likewise for I' . We additionally have image features from the image descriptor which we can sample for feature information to get features f_i, f'_i .

Given these sparse and unfiltered correspondences, we look to estimate a function to interpolate this deformation. The model should use only relevant features to estimate the interpolation. First we learn to discard outlier correspondences with match 'refinement' which adjusts the match locations to better fit the correct underlying deformation, moving outliers to reasonable deformations. This is done using a graph attention style network [34] with edges between each point and its k nearest detected points. After we have refined the match positions, we use these new positions and the learned graph features we obtain alongside them to estimate tissue movement at any query position. We approach the movement estimation by creating a neural implicit representation [16] and estimating the movement of a query point as a function of its neighboring correspondences. Our implementation, KINFlow, is a novel application of implicit representations which allows for selective tracking of relevant points without having to densely estimate full image transformations. That said, our method can still track motion at full resolution, or even upsample to a higher resolution.

A. Match Filtering and Refinement

We first take in the coarse set of matches between frames I, I' , and estimate new feature descriptors for all matches, along with refined displacements, outputting $(h_i, d'_i)_{i \in \mathcal{V}}$. The refinement model uses the point location, relative motion, and descriptor of each match, embedding the positional features using a positional encoding layer, denoted as ϕ . This passes the position through multiple sinusoids of varying frequency [16], either in 1D (distance) or 2D (position, displacement) for our case. For this paper, γ denotes a linear layer followed by a ReLU activation function. Repetitions of ϕ and γ each count as new instantiations with different weights.

$$g_i = \phi(p_i) + \phi(p'_i) + \gamma(f_i) + \gamma(f'_i) + \phi(\text{dist}(p_i, p'_i)) \quad (1)$$

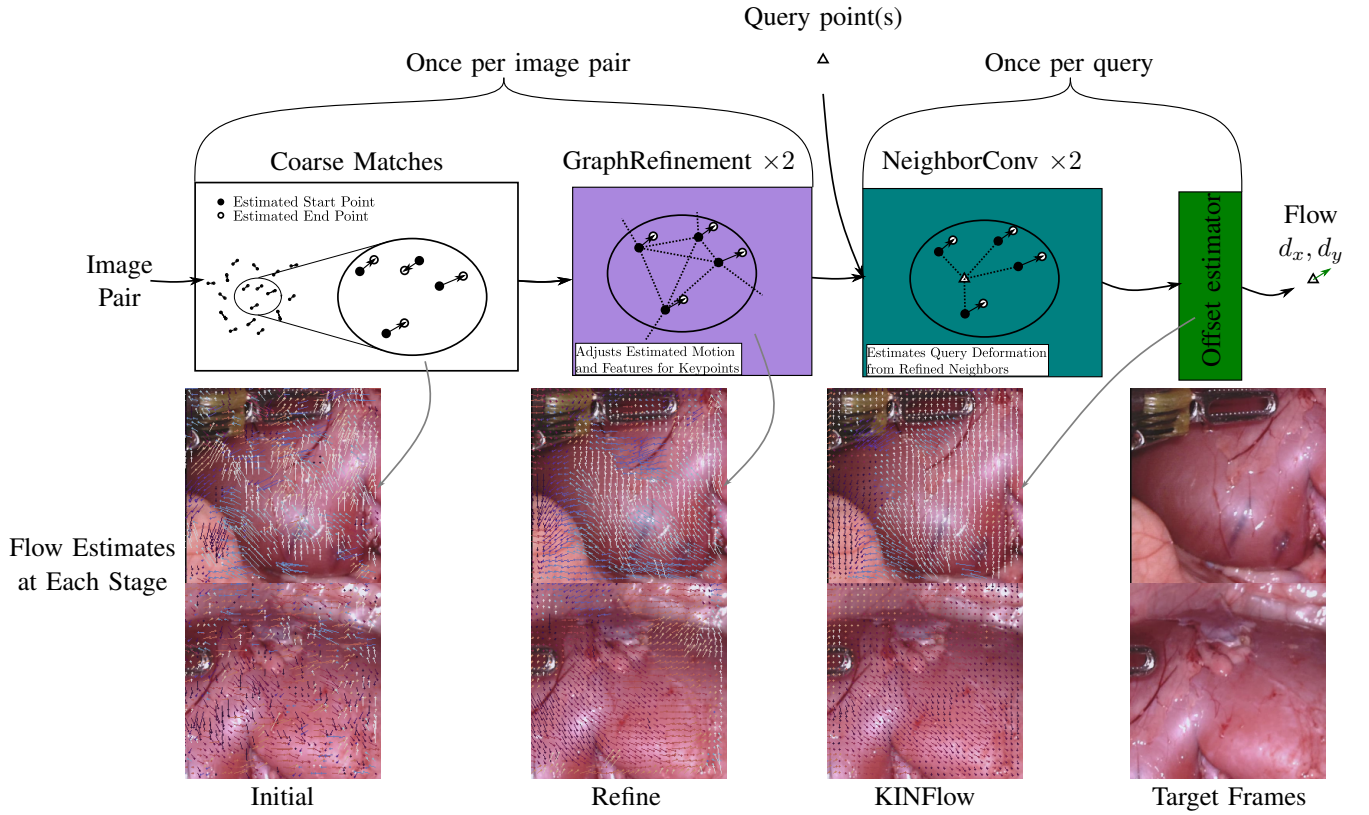


Fig. 1: Our procedure for dense estimation of flow at any query point. Our model takes in unrefined nearest neighbor matches and their features (Initial), then we pass these matches through a graph neural network to obtain refined positions and features (Refine); finally for each query point, we create a graph consisting solely of its neighbors and use its position and our NeighborConv for estimating a flow field at the query point (KINFlow). The images are flow fields of a grid of queries evaluated at each step in the process, with the coloring of each vector representing angle of movement.

Now, that we have the embedded feature for all vertices $\mathbf{g} = \{g_1, g_2, \dots, g_N\}, g_i \in \mathbb{R}^c$, we can pass information from each detected point to its neighbors using graph attention layers.

Our graph neural network with attention learns refined displacement values $\mathbf{d}'$, along with the new features, $\mathbf{h}$ for each correspondence. This allows each match to take into account the matches in the neighboring region for generating its new features. This is similar to the graph attention transformer, the set transformer or similar works [34], [35], [48], but we are additionally integrating an embedding for each point according to its position to allow the network to treat points according to their relative position. Each keypoint is a node in the graph, with the neighbors (in 2D position) denoted as $\mathcal{N}_i$. Edges are chosen to create a k-nearest neighbor graph, $\mathcal{E} = \{(i, j) | i \in \mathcal{V}, j \in \mathcal{N}_i\}$. This graph is calculated for each new frame according to the detected positions p_i . Our graph attention operation we use takes in a single feature key $k \in \mathbb{R}^c$, and a set of values $\mathbf{v} \in \mathbb{R}^{c \times k}$, mixing the values of $\mathbf{v}$ together. We use this operation for both graph convolutions and our neighbor

convolution.

$$\text{Attn}(k, \{v_1, v_2, \dots, v_k\}) = \sum_j \alpha_j v_j \quad (2)$$

$$\alpha_j = \text{softmax}_j(\mathbf{W}k \cdot \mathbf{W}v_j) \quad (3)$$

First we embed neighbors the neighbors of each keypoint into a set $\mathbf{m}_i$ of k points for each vertex, i .

$$\mathbf{m}_i = \{g_j + \phi(p_i - p_j) | j : (i, j) \in \mathcal{E}\} \quad (4)$$

$$\text{GraphConv}(\mathbf{g}) = \gamma(\text{Attn}(g_i, \mathbf{m}_i)) + \gamma(g_i) \quad (5)$$

If the environment were rigidly transformed, we could use a RANSAC style scheme for removal of outliers. We have non-rigid deformation, leaving us with some options such as SuperGlue [24] or GMSMatch [49]. Instead we look to refine and estimate features instead of just match. This also enables the network to choose to discount outlier features. If we stack two *GraphConv* layers we have a set of features that can be used downstream for flow, but also for refined match estimates. The output features of the two *GraphConv* layers are $\mathbf{h} = \{h_1, h_2, \dots, h_N\}, h_i \in \mathbb{R}^c$. We estimate a refined flow for each vertex as $d'_i = \text{linear}(\gamma(h_i))$. At this stage for each detected point we have an updated flow estimate along with a feature that will be used for flow interpolation.

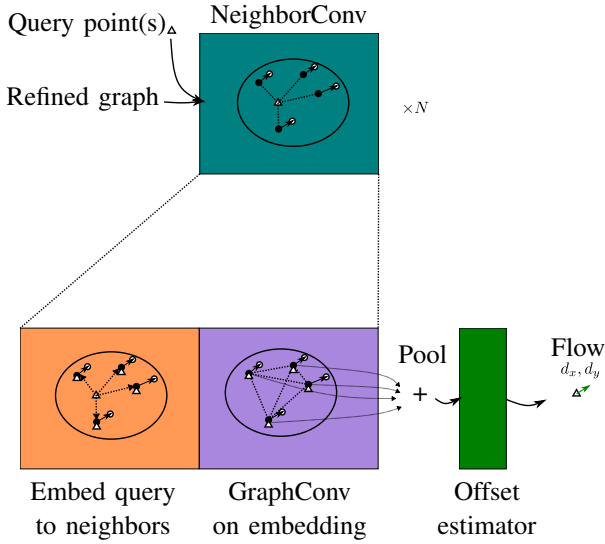


Fig. 2: KINFlow (k-nearest neighbor implicit neural flow). In short we broadcast features between the query point and its neighbor to each respective neighbor. After broadcasting, we run a *GraphConv* layer. To get a final estimate, we pool features with a mean.

B. KINFlow

Given a reasonable estimate of surrounding correspondences, we would like to move the points accordingly. Prior methods do this with splines, embedded deformation, etc. We instead opt to treat a point's movement as a direct function of the learned movement of its neighbors.

We model the movement of any point in the image using an implicit neural representation. For each query, its flow estimate depends only on a local graph of k neighbors. We take information from the query point corresponding to each neighbor and concatenate it to each of the neighbor keypoints. Then we pass it through a *GraphConv*, and follow up with a pooling operation at the end to estimate the query flow. This is effectively an implicit neural representation conditioned on the neighbors in the graph. See Fig. 2 for an illustration. The neighbor lookup could be optimized in a dense manner by rendering each neighbor to its the k -voronoi diagrams or using a k -d tree, but for our current purposes we look them up with brute force search.

For each query point, we take a k -NN graph, and pool the point onto the graph. We positionally embed the relative position and distance too so the network has the option to treat them by distance (ReLU's are less good at this, often smoothing frequencies [15], [16])

$$\mathbf{w}_q = \{w_{qi}\}_{i \in [1, k]} = ((\phi(p_v), \phi(d'_v), h_v))_{v \in \mathcal{N}_q} \quad (6)$$

This set of points is all that is required in estimating the flow. This frees us of reliance on long-range dependencies in the flow estimation step, as the far-off information has been accumulated nearby during the refinement step.

For each set of points $\mathbf{w}_q$ in a neighborhood of query point q , we add some information from q to each neighbor,

so they hold information of where the query is relative to it. If a point is far, it should have less influence and be able to learn that. We abuse notation, with indexing by w_{qi} denoting gathering the keypoint's relevant feature from neighbor i of query q .

$$x_{qi} = w_{qi} + \phi(\text{dist}(p_{w_{qi}}, q)) + \phi(p_{w_{qi}} - q) + \phi(q) + \gamma(f_q) \quad (7)$$

f_q is the image descriptor feature at location q , and this allows the network to both receive the query position and its image feature (eg. the network may not want to influence an instrument point query with a tissue point neighbor). That is, each neighbor is a combination of its own information along with its relative position to the query, the query location, and the query feature. This graph is a k -clique, that passes information between all of the query neighbors. The features are the set of embedded queries, $\mathbf{x}_q = \{x_{qi}\}_{i \in [1, k]}$

$$\text{NeighborConv}(q) = \text{GraphConv}(\mathbf{x}_q)$$

We take these descriptors and run them through attentional layers, using the same *GraphConv* operation in 5 that uses attention between each keypoint. We repeat this convolution twice, with different weights. Finally, we compress these together with a sum, going from k vectors of size c to a single feature vector of size c . This feature vector is then passed through a residual layer and a convolution, resulting in the 2D estimate of flow, with d_q as the flow of query q

$$d_q = \text{linear}(\gamma(\frac{1}{k} \sum_i \text{GraphConv}(\text{GraphConv}(x_{qi})))) \quad (8)$$

IV. EXPERIMENTS

For training our model, we use an internally collected in-house dataset for surgical training from Intuitive Surgical. The dataset has been captured in porcine clinical training labs where surgeons were being trained on the Xi surgical system by completing advanced clinical training tasks. The Xi system endoscope is factory calibrated and Intuitive has provided all calibration data for all of the scopes being used for this stereo-video dataset. This dataset includes camera and instrument motion, suturing, and other transitions, with 8 separate rectified stereo videos at resolution of 1280×1080 . We select two videos for testing, which provides 275,534 training frames and 90,402 possible test frames. We random sample frames from the test set for experiments to make testing more efficient. The videos average 12.5 minutes in length at a rate of 30 frames per second. Some example frames are in Fig. 1. First, we train the ReTRo [45] descriptor on this data. This descriptor requires relative camera position for training, for which we use the stereo transformation between left and right images as the weak supervision. After training the descriptor, we train our flow model. As many unsupervised methods do, we use a mixture of L1 and Structural Similarity Index Measure (SSIM) for an image based loss [50]:

$$\mathcal{L}_{im} = \alpha \frac{1 - \text{SSIM}(I, I')}{2} + (1 - \alpha) \|I - I'\| \quad (9)$$

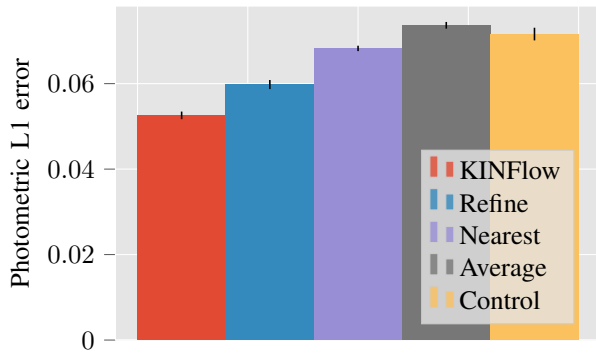


Fig. 3: Average photometric error for each type of flow estimation using random video frame pairs from the two sequences in our test set. Our neural interpolation method outperforms our refinement, and just direct use of descriptors.

We set $\alpha = 0.9$, and I, I' are the warped initial frame and the end frame respectively.

We train by sampling random image crops of size 256×256 from our training set, and use PyTorch with a learning rate of 10^{-4} . For the graph refinement step we use a $k = 8$ neighbors, and for flow querying a $k = 4$. All convolutions/graph operations have channels $c = 64$. We generate 256-dimensional ReTRO descriptors, and detect $N = 512$ FAST keypoints per crop. We match these with their nearest descriptor in the comparison image. For our flow and refinement network, we use random samples of image pairs at varying frame distance selected uniformly from 15 to 30 frames apart in a video. This enables a reasonable range of image changes, allowing the model to see different rates of change from one-half to one second.

We train for 100,000 steps on a 1080Ti which takes around 12 hours. While training, we densely run our model to estimate flow, creating a query point for every pixel in the image. This enables us to have as much gradient information as possible for each step. Although we train in a dense manner to allow full use of image information and reduction of noise, our model can evaluate sparsely at query points.

We test multiple different methods: KINFlow, which is our method that estimates flow using neural implicit functions for each query point; Refine, which estimates flow by using the flow of the nearest refined correspondence from our graph refinement; Nearest, which estimates flow using the non-refined nearest neighbor; Average, which estimates flow by simply averaging the movement of the four nearest non-refined neighbors; and Control which estimates zero movement. To quantify our results, we densely estimate flow, and use our method to deform I to I' with bilinear interpolation. We quantify our results by looking at the L1 error between these images.

A. Results on our test set

For the test results, we randomly select 1024 temporal image pairs with varying frame skips of the same size as the training set. In Fig. 3 we calculate the L1 difference

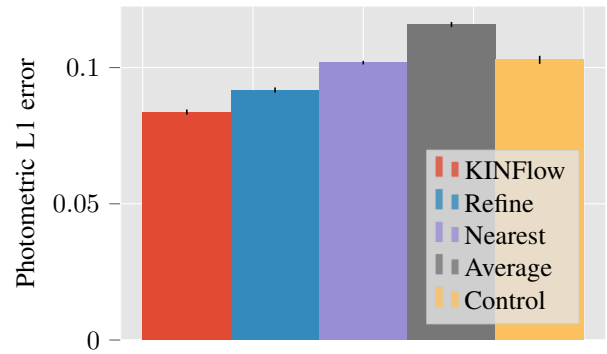


Fig. 4: Results on the Hamlyn dataset [51], with different endoscope. Our method is still competitive.

between images, and plot the average difference along with their standard error. The Intuitive Surgical dataset includes a large amount of transitions, camera motions, changes of imaging modality from visual to ICG, and instrument motion.

As can be seen in Fig. 3, KINFlow outperforms interpolation using point refinement, no refinement, and averaging for refinement. For a demonstration of how the flow looks qualitatively on the test set, refer to Fig. 1. For photometric error, the control case of zero movement estimate sometimes outperforms the simple methods, as low movement can provide a relatively good estimate. Especially compared to a method such as averaging that will be skewed heavily by errors.

B. Generalization to other data

Although we train on a dataset with a particular resolution and scenes, we expect our model to perform well on different datasets in surgery. Rather than depending directly on image features, KINFlow uses image features and their estimated movement to interpolate flow, operating on a graph instead of on a fixed resolution. Thus we show our models performance on two different types of data.

We test our method on the Hamlyn dataset for stereo estimation, which has a different scene (partial nephrectomy) and camera type. This data was collected from a da Vinci Si surgical system, and has a resolution of 384×192 [51]. Without any training or tuning of either the descriptor or our flow model on this test set, KINFlow performs well as seen in Fig. 4. This method performs better throughout, with a significant margin in the ranking between KINFlow as best, refinement as second, and nearest neighbor interpolation coming up third. For a qualitative view, look at Fig. 5 for our flow model compared to using nearest refined matches.

We then test on the dataset of EndoSLAM. This dataset includes video from ex-vivo environments created using real tissue. Their data has ex-vivo non-surgical video sequences from reconstructed scenes of stomach, small intestine, and colon. This dataset is introduced for testing rigid mapping and depth estimation methods, but it still provides a useful way for us to test our method as it includes camera motion. Thus we can see how well our method works for tracking in

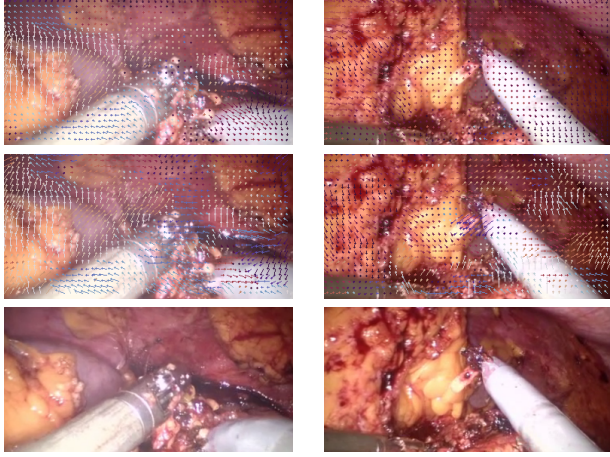


Fig. 5: Images from running KINFlow on the Hamlyn dataset [51], showing generalization to other data qualitatively. Top: KINFlow, middle: Refine, bottom: next frame. Note how KINFlow removes discontinuities. Color of vectors represent the angle of motion.

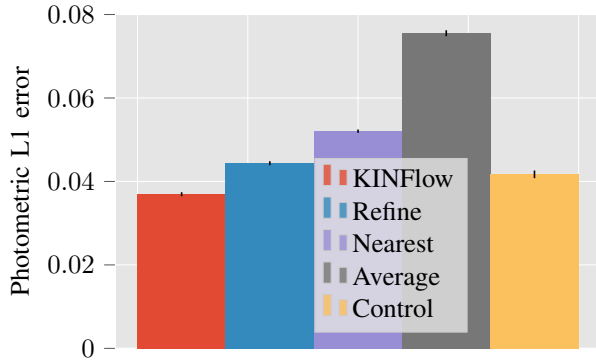


Fig. 6: Comparison results of our method on the EndoSLAM [52] dataset. This dataset has camera motion through multiple ex-vivo environments with multiple different camera types.

this case. We use data from their cameras HighCam 1280 x 720 and LowCam 640 x 480 in our testing, random sampling temporal pairs. Using KINFlow for estimation still keeps the place of lowest error as shown in Fig. 6. Of additional note, the control method has better performance in this case than in the other test sets due to the average motion in these scenes being smaller for the test data.

C. Forward-Backward Model

We initially attempted implementing a model that takes in the refinement features and outputs from the prior frame pair in addition to the current one for the purpose of improving flow estimation, but did not achieve any statistically significant difference in results. Likely due to the fact that two sparse sets of motion correspondence over time is not enough for the model to improve its estimate in dense regions. Branching off of the same idea, we decided to implement a method that runs correspondence refinement from I to I'

TABLE I: Comparison of using forward graph refinement versus using both forward and backward graph refinement.

Model	Mean	Standard Error
Forward	0.0534	0.00087
ForwardBackward	0.0526	0.00089

TABLE II: Average time over 32 runs for graph refinement and flow queries. $\pm$ denotes standard error of mean.

Keypoints	Queries	Refinement Time	Flow Querying Time
512	1024	0.636 ± 0.017 ms	1.90 ± 0.022 ms
512	4096	0.638 ± 0.018 ms	7.27 ± 0.043 ms
512	16384	0.641 ± 0.015 ms	30.26 ± 0.142 ms
1024	1024	1.41 ± 0.013 ms	2.04 ± 0.015 ms
1024	4096	1.44 ± 0.013 ms	7.97 ± 0.036 ms
1024	16384	1.44 ± 0.014 ms	33.12 ± 0.111 ms

in parallel with doing the correspondence in reverse from I' to I . This leaves us with two sets of displacements, and additionally two feature sets. The reverse one should be close to the sign-flip of the other, but if it is not, it could act as a signal for poor correspondence. We input both of these into the flow model by concatenating the embedded features from the reverse refinement to the closest detected points in the forward correspondence. This has better results at the cost of having to run graph refinement twice. Mean photometric error in each model is summarized in Table I.

D. Timing

For our method to track 1024 points without any optimization using 512 keypoints as correspondences (as shown in Fig. 1), it takes 0.6 ms for point refinement, and 1.9 ms to query the flow points. We measure this by checking time before and after synchronizing cuda operations on a 1080 Ti. Note we are not timing the descriptor creation or initial matching. Timing begins immediately after image descriptor calculation. All experiments were run with a fixed batch size of 8. We run each experiment 32 times to provide a reliable estimate. Table II demonstrates our benchmarking results over multiple runs of different sizes. Increasing the number of queries increases the time by a proportional amount.

V. CONCLUSION

We show that KINFlow outperforms other methods for fast interpolation of keypoint correspondences in addition to having good generalization ability for different types of data. KINFlow is fast, and will help enable methods for surgical perception and image guidance without the need to solve for a deformation model or force a prior onto the environment. We expect the resolution and prior-free interpolation method of KINFlow to be useful in other domains that have deformable and changing data. Future work will quantify this performance in labs and experiments to analyze physical error in mm or pixels.

REFERENCES

- [1] S. Giannarou, M. Ye, G. Gras, K. Leibrandt, H. Marcus, and G. Yang, "Vision-based deformation recovery for intraoperative force estimation of tool-tissue interaction for neurosurgery," *IJCARS*, 2016.
- [2] R. Richa, A. P. Bó, and P. Poignet, "Towards robust 3D visual tracking for motion compensation in beating heart surgery," *Medical Image Analysis*, 2011.
- [3] H. Zhou and J. Jayender, "Real-time Nonrigid Mosaicking of Laparoscopy Images," *IEEE Trans. Med. Imaging*, pp. 1–1, 2021.
- [4] Y. Li, F. Richter, J. Lu, E. K. Funk, R. K. Orosco, J. Zhu, and M. C. Yip, "SuPer: A Surgical Perception Framework for Endoscopic Tissue Manipulation with Surgical Robotics," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2294–2301, Apr. 2020.
- [5] Z. Fu, Z. Jin, C. Zhang, Z. He, Z. Zha, C. Hu, T. Gan, Q. Yan, P. Wang, and X. Ye, "The Future of Endoscopic Navigation: A Review of Advanced Endoscopic Vision Technology," *IEEE Access*, 2021.
- [6] P. Mountney and G.-Z. Yang, "Soft Tissue Tracking for Minimally Invasive Surgery: Learning Local Deformation Online." Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 364–372.
- [7] D. Stoyanov, G. P. Mylonas, F. Deligianni, A. Darzi, and G. Z. Yang, "Soft-Tissue Motion Tracking and Structure Estimation for Robotic Assisted MIS Procedures." Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 139–146.
- [8] M. C. Yip, D. G. Lowe, S. E. Salcudean, R. N. Rohling, and C. Y. Nguan, "Real-Time Methods for Long-Term Tissue Feature Tracking in Endoscopic Scenes," in *IPCAI*, 2012.
- [9] S. Giannarou, M. Visentini-Scarzarella, and G. Yang, "Probabilistic Tracking of Affine-Invariant Anisotropic Regions," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 130–143, Jan. 2013.
- [10] A. Schoob, D. Kundrat, L. A. Kahrs, and T. Ortmaier, "Stereo vision-based tracking of soft tissue motion with application to online ablation control in laser microsurgery," *Medical Image Analysis*, vol. 40, pp. 80–95, Aug. 2017.
- [11] J. Song, J. Wang, L. Zhao, S. Huang, and G. Dissanayake, "MIS-SLAM: Real-Time Large-Scale Dense Deformable SLAM System in Minimal Invasive Surgery Based on Heterogeneous Computing," *IEEE Robot. Autom. Lett.*, Oct. 2018.
- [12] J. Lu, A. Jayakumari, F. Richter, Y. Li, and M. C. Yip, "SuPer Deep: A Surgical Perception Framework for Robotic Tissue Manipulation using Deep Learning for Feature Extraction," *ArXiv200303472 Cs*, Sep. 2020.
- [13] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," *ArXiv200312039 Cs*, Aug. 2020.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," *ArXiv200308934 Cs*, Mar. 2020.
- [15] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, "Implicit Neural Representations with Periodic Activation Functions," *NeurIPS*, 2020.
- [16] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, "Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains," *NeurIPS*, 2020.
- [17] T. Sun, K.-E. Lin, S. Bi, Z. Xu, and R. Ramamoorthi, "NeLF: Neural Light-transport Field for Portrait View Synthesis and Relighting," *ArXiv210712351 Cs*, Jul. 2021.
- [18] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, "pixelNeRF: Neural Radiance Fields from One or Few Images," *ArXiv201202190 Cs*, Dec. 2020.
- [19] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-NeRF: Neural Radiance Fields for Dynamic Scenes," *ArXiv201113961 Cs*, Nov. 2020.
- [20] L. Yen-Chen, P. Florence, J. T. Barron, A. Rodriguez, P. Isola, and T.-Y. Lin, "iNeRF: Inverting Neural Radiance Fields for Pose Estimation," *ArXiv201205877 Cs*, Dec. 2020.
- [21] D. Azinovic, R. Martin-Brualla, D. B. Goldman, M. Nießner, and J. Thies, "Neural RGB-D Surface Reconstruction," p. 12.
- [22] Y. Deng, J. Yang, and X. Tong, "Deformed Implicit Field: Modeling 3D Shapes with Learned Dense Correspondence," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Nashville, TN, USA: IEEE, Jun. 2021, pp. 10 281–10 291.
- [23] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit Mapping and Positioning in Real-Time," *ArXiv210312352 Cs*, Mar. 2021.
- [24] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "Super-Glue: Learning Feature Matching With Graph Neural Networks," in *CVPR*, 2020.
- [25] A. Sinha, Z. Murez, J. Bartolozzi, V. Badrinarayanan, and A. Rabinovich, "DELTA: Depth Estimation by Learning Triangulation and densification of Sparse points," *ArXiv200308933 Cs Eess*, Aug. 2020.
- [26] S. Jiang, Y. Lu, H. Li, and R. Hartley, "Learning Optical Flow from a Few Matches," in *CVPR*. Nashville, TN, USA: IEEE, 2021, pp. 16 587–16 595.
- [27] K. He, Y. Zhao, Z. Liu, D. Li, and X. Ma, "Whole-pixel registration of non-rigid images using correspondences interpolation on sparse feature seeds," *Vis Comput*, Mar. 2021.
- [28] A. Božič, P. Palafox, M. Zollhöfer, J. Thies, A. Dai, and M. Nießner, "Neural Deformation Graphs for Globally-consistent Non-rigid Reconstruction," *CVPR*, 2021.
- [29] A. Bozic, P. Palafox, M. Zollhöfer, A. Dai, J. Thies, and M. Niessner, "Neural Non-Rigid Tracking," in *NeurIPS*, 2020.
- [30] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *CVPR*, 2017.
- [31] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *NeurIPS*, 2017.
- [32] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic Graph CNN for Learning on Point Clouds," *ACM Trans. Graph.*, 2019.
- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All you Need," in *NeurIPS*, 2017.
- [34] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio', and Y. Bengio, "Graph Attention Networks," *ICLR*, 2018.
- [35] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. Teh, "Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks," *undefined*, 2019.
- [36] P. Erler, P. Guerrero, S. Ohrhallinger, N. J. Mitra, and M. Wimmer, "Points2Surf Learning Implicit Surfaces from Point Clouds," in *ECCV*, 2020.
- [37] Z. Wang, P. Wang, Q. Dong, J. Gao, S. Chen, S. Xin, and C. Tu, "Neural-IMLS: Learning Implicit Moving Least-Squares for Surface Reconstruction from Unoriented Point clouds," 2021.
- [38] U. Wickramasinghe, G. Knott, and P. Fua, "Deep Active Surface Models," *ArXiv201108826 Cs*, Jun. 2021.
- [39] M. C. Yip, D. G. Lowe, S. E. Salcudean, R. N. Rohling, and C. Y. Nguan, "Tissue Tracking and Registration for Image-Guided Surgery," *IEEE Trans. Med. Imaging*, vol. 31, no. 11, pp. 2169–2182, Nov. 2012.
- [40] J. Zhan, J. Cartucho, and S. Giannarou, "Autonomous Tissue Scanning under Free-Form Motion for Intraoperative Tissue Characterisation," *2020 IEEE Int. Conf. Robot. Autom. ICRA*, 2020.
- [41] J. J. G. Rodríguez, J. Lamarca, J. Morlana, J. D. Tardós, and J. M. M. Montiel, "SD-DefSLAM: Semi-Direct Monocular SLAM for Deformable and Intracorporeal Scenes," *ArXiv201009409 Cs*, Oct. 2020.
- [42] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Key-points," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004.
- [43] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded Up Robust Features," in *Computer Vision – ECCV 2006*, A. Leonardis, H. Bischof, and A. Pinz, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, vol. 3951, pp. 404–417.
- [44] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *2011 International Conference on Computer Vision*, 2011.
- [45] A. Schmidt and S. E. Salcudean, "Real-Time Rotated Convolutional Descriptor for Surgical Environments," in *MICCAI*, 2021.
- [46] Q. Wang, X. Zhou, B. Hariharan, and N. Snavely, "Learning Feature Descriptors Using Camera Pose Supervision," in *Computer Vision – ECCV 2020*, 2020.
- [47] M. J. Tyszkiewicz, P. Fua, and E. Trulls, "DISK: Learning local features with policy gradient," *ArXiv200613566 Cs*, Jun. 2020.
- [48] F. Chen, S. Pan, J. Jiang, H. Huo, and G. Long, "DAGCN: Dual Attention Graph Convolutional Networks," *2019 Int. Jt. Conf. Neural Netw. IJCNN*, 2019.
- [49] J.-W. Bian, W.-Y. Lin, Y. Liu, L. Zhang, S.-K. Yeung, M.-M. Cheng, and I. Reid, "GMS: Grid-Based Motion Statistics for Fast, Ultra-robust Feature Correspondence," *Int. J. Comput. Vis.*, vol. 128, pp. 1580–1593, Jun. 2020.

- [50] C. Godard, O. M. Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *CVPR*, 2017.
- [51] M. Ye, E. Johns, A. Handa, L. Zhang, P. Pratt, and G.-Z. Yang, "Self-Supervised Siamese Learning on Stereo Image Pairs for Depth Estimation in Robotic Surgery," *ArXiv170508260 Cs*, May 2017.
- [52] K. B. Ozyoruk, G. I. Gokceler, G. Coskun, K. Incetan, Y. Almalioglu, F. Mahmood, E. Curto, L. Perdigoto, M. Oliveira, H. Sahin, H. Araujo, H. Alexandrino, N. J. Durr, H. B. Gilbert, and M. Turan, "EndoSLAM Dataset and An Unsupervised Monocular Visual Odometry and Depth Estimation Approach for Endoscopic Videos: Endo-SfMLearner," *ArXiv200616670 Cs*, Oct. 2020.